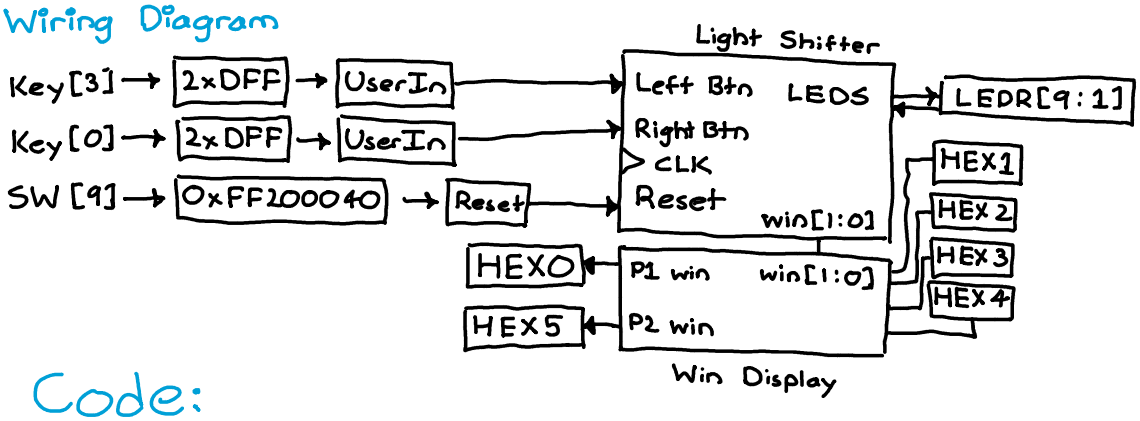
Lab 4

Thursday, July 31, 2025 1:43 PM

Wiring Diagram



Code:

```
module DE1_SoC (CLOCK_50, HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, KEY, LEDR, SW);

   input logic CLOCK_50; // 50MHz clock.
   output logic [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5;
   output logic [9:0] LEDR;
   input logic [3:0] KEY; // True when not pressed, False when pressed
   input logic [9:0] SW;

   // Auxillary variables
   logic [1:0] winner;
   logic [8:0] out;
   logic reset;

   // Assign old values
   assign reset = SW[9];

   // Might need to check later
   light_shifter s (.clk(CLOCK_50), .reset, .left_btn(KEY[3]), .right_btn(KEY[0]), .leds(out), .winner);

   TugOfWarHex t (.HEX0, .HEX1, .HEX2, .HEX3, .HEX4, .HEX5, .winner);

   // Show signals on LEDRs so we can see what is happening
   assign LEDR[9:1] = out;
   assign LEDR[0] = 1'b0;

endmodule
```

```
module light_shifter (
    input  logic clk,
    input  logic reset,
    input  logic left_btn,
    input  logic right_btn,
    output logic [9:1] leds,
    output logic [1:0] winner
);

    // LED register - now 9 bits
    logic [9:1] led_reg;

    // Edge detection
    logic left_prev, right_prev;
    logic left_rising, right_rising;

    // --- Edge Detection ---
    always_ff @(posedge clk) begin
        if (reset) begin
            left_prev  <= 1'b1;
            right_prev <= 1'b1;
        end else begin
            left_prev  <= left_btn;
            right_prev <= right_btn;
        end
    end

    assign left_rising  = ~left_btn  & left_prev;
    assign right_rising = ~right_btn & right_prev;

    // --- Shift Logic ---
    always_ff @(posedge clk) begin
        if (reset) begin
            led_reg <= 9'b000010000;  // Start with bit 0 ON
            winner <= 2'b00;
        end else begin

            if (left_rising && ~right_rising) begin

                  if (led_reg[9]) begin
                     winner <= 2'b10;
                  end


                led_reg <= led_reg << 1;    // Shift left
            end else if (right_rising & ~left_rising) begin

                  if (led_reg[1]) begin
                     winner <= 2'b01;
                  end


                led_reg <= led_reg >> 1;    // Shift right

            end
        end
    end

    assign leds = led_reg;

endmodule
```

```
module TugOfWarHex (HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, winner);

   output logic [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5;
   input logic [1:0] winner;

   always_comb begin
      case (winner)
         2'b10: begin
            HEX0 = 7'b1111111;  // player 1 wins
            HEX1 = 7'b1111111;
            HEX2 = 7'b1111111;
            HEX3 = 7'b1111111;
            HEX4 = 7'b1111111;
            HEX5 = 7'b1111001;

         end

         2'b01: begin
            HEX0 = 7'b0100100;  // player 2 wins
            HEX1 = 7'b1111111;
            HEX2 = 7'b1111111;
            HEX3 = 7'b1111111;
            HEX4 = 7'b1111111;
            HEX5 = 7'b1111111;
         end

         default: begin
            HEX0 = 7'b1111111;
            HEX1 = 7'b1111111;
            HEX2 = 7'b1111111;
            HEX3 = 7'b1111111;
            HEX4 = 7'b1111111;
            HEX5 = 7'b1111111;
         end
      endcase
   end
endmodule
```

Resources

	Compilation Hierarchy Node	Combinational ALUTs	Dedicated Logic Registers
1	⌄ |DE1_SoC	19 (0)	13 (0)
1	|TugOfWarHex:t|	2 (2)	0 (0)
2	|light_shifter:s|	17 (17)	13 (13)

Score Calculation:

ALUTs + Logic Registers = 19 + 13

= 32